

Parallelism (PAR)

Unit 1: Why parallel computing?

Eduard Ayguadé,
Gladys Utrera (T20) and Daniel Jiménez (T10, T40)

Computer Architecture Department
Universitat Politècnica de Catalunya

Course 2025/26 (Spring semester)

Learning material for this lesson

- ▶ Atenea: Unit 1 "Why parallel computing?"
 - ▶ Video lesson 1: serial vs. parallel
 - ▶ Questions after video lesson 1
 - ▶ Going further: dining philosophers problem
- ▶ These slides to dive deeper into the concepts in Unit 1
- ▶ Collection of Exercises: problems in Chapter 1

Outline

Motivation (video lesson 1)

Concurrency and parallelism

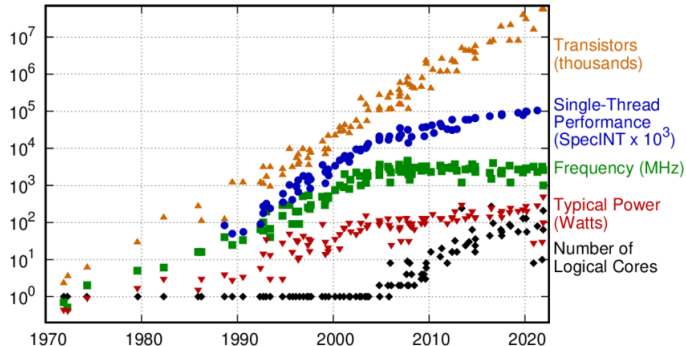
Examples and potential problems

Tasks, Threads, Processes and Processors

Concepts in video lesson 1

- ▶ Serial execution on 1 processor
 - ▶ Instructions are executed one after another, only one at any moment in time
 - ▶ $T = N \div F$, to run faster one should increase F (number of instructions per second executed)
 - ▶ How? $F = IPC \times freq$, being IPC the number of instructions per cycle and $freq$ the frequency of the processor or number of cycles per second $\rightarrow$ architecture and technology
- ▶ Parallel execution on P processors
 - ▶ Each processor executes $\frac{1}{P}$ instructions of the program
 - ▶ $T = (N \div P) \div F$
- ▶ Throughput computing on P processors
 - ▶ k programs executed on P processors
 - ▶ OS multiplexes their execution on time, fairness

Uniprocessor and multicore performance evolution



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Uniprocessor and multicore performance evolution (cont.)

- ▶ Moore's law: the number of transistors on an integrated circuit potentially doubles approximately every two years
 - ▶ Higher clock frequencies
 - ▶ More complex architectural designs (ability to exploit Instruction-Level Parallelism, ILP)
- ▶ But ...
 - ▶ Diminishing returns when using transistors to exploit more ILP
 - ▶ Power consumption/heat dissipation: hard technological limit
- ▶ As an alternative to scale performance, each generation of Moore's law allows to potentially double the number of cores
 - ▶ This vision creates a desperate need for all computer scientists and practitioners to be aware of parallelism¹

¹Parallelism and parallel computing has been taught for several decades in some master and PhD curricula, oriented to solve computationally intensive applications in science and engineering with problems too large to solve on one computer.

Scaling beyond multicores ... servers ... supercomputers

Top500.org ranking the most powerful supercomputers (11/2025)

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, QuadRail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
...					
14	MareNostrum 5 ACC - BullSequana XH3000, Xeon Platinum 8460Y+ 32C 2.3GHz, NVIDIA H100 64GB, Infiniband NDR, EVIDEN EuroHPC/BSC Spain	663,040	175.30	249.44	4,159
...					

Give me an example of their use!

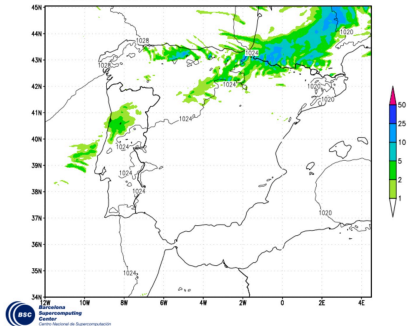
Rainfall 60 hour forecast		
Machine	Parallel	Sequential
MN3	32 min (128 cores)	2.5 days approx.
MN4	23 min (128 cores)	

Figure 1: Accumulated rainfall

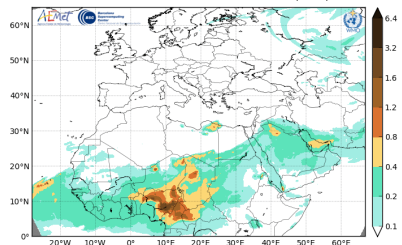
Dust Disp. 70 hour forecast		
Machine	Parallel	Sequential
MN3	49 min (260 cores)	8 days approx.
MN4	32 min (248 cores)	

Figure 2: Sahara dust dispersion

BSC-ES/FORECAST WRF-ARWv3.5.1 6h acc. Precipitation (mm) and MSL pressure (hPa)
36h forecast for 06UTC 17 FEB 18 - Iberian Peninsula Res:4x4km



Barcelona Dust Forecast Center - <http://dust.aemet.es/>
NMMB/BSC-Dust Res:0.1°x0.1° Dust AOD
Run: 12h 15 FEB 2018 Valid: 12h 15 FEB 2018 (H+00)



Outline

Motivation (video lesson 1)

Concurrency and parallelism

Examples and potential problems

Tasks, Threads, Processes and Processors

Concurrent execution

Exploiting concurrency consists in breaking a problem into discrete parts, to be called tasks, to ensure their correct simultaneous execution

- ▶ Each (serial) task is sequentially executed on a single CPU ...
- ▶ ... but multiple tasks interleave their execution on the CPU, so that the CPU time is multiplexed among tasks. The SO is in charge of preempting the currently running task, for example after its time quantum expires, starting or resuming the execution of another task

Need to manage and coordinate the execution of tasks, ensuring correct access to shared resources

Example: client/server application

Client connection implies the execution of the client task (C). As a response, the server task (T) is executed

Task C_k: receives client requests (duration 1 time unit)

Task T_k: executes a single bank transaction (e.g. withdraw/deposit some money in bank account, with a duration of 4 time units)

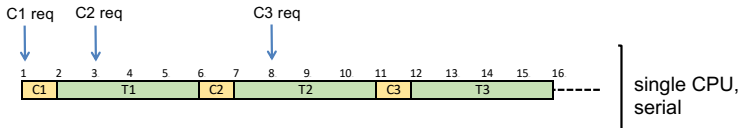


Figure 3: Sequential execution of client and server tasks

Example: client/server application

Client connection implies the execution of the client task (C). As a response, the server task (T) is executed

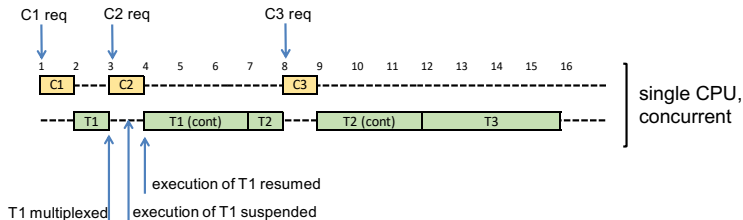


Figure 4: Concurrent execution of client and server tasks, but server tasks serialized

Example: client/server application

Client connection implies the execution of the client task (C). As a response, the server task (T) is executed

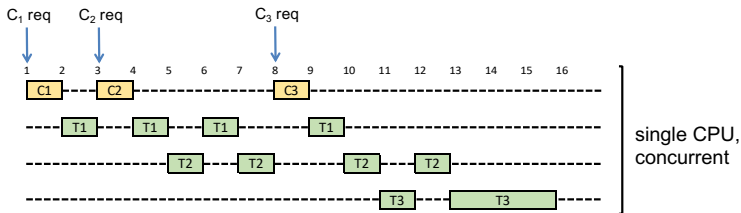


Figure 5: Concurrent execution of client and multiple server tasks

Parallel execution

Parallel execution: to reduce the execution (response) time of a program. Parallelism is when we use multiple processors (CPU) to simultaneously execute the tasks identified for concurrent execution

- ▶ 1 program on p processors
- ▶ Ideally, each CPU could receive $\frac{1}{p}$ of the program, reducing its execution time by p

Example: client/server application

Client connection implies the execution of the client task (C). As a response, the server task (T) is executed

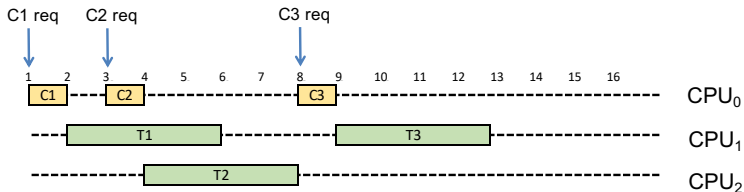


Figure 6: Parallel execution of client and server tasks on **several processors**

Throughput vs. parallel computing

Throughput computing: multiple processors can also be used to increase the number of programs executed per time unit

- ▶ *Multiprogrammed* execution of multiple, unrelated, instruction streams (programs) at the same time on multiple processors
- ▶ n programs on p processors; if $(n \geq p)$ each program receives $\frac{p}{n}$ processors, one processor otherwise

Outline

Motivation (video lesson 1)

Concurrency and parallelism

Examples and potential problems

Tasks, Threads, Processes and Processors

Examples and potential problems

bank (hash map)

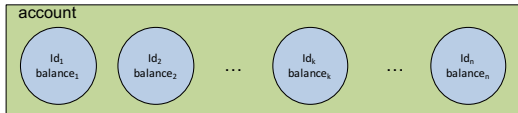


Figure 7: Bank with several accounts

Three different cases and potential problems:

- ▶ Example 1: two simultaneous deposit/withdraw operations
 - ▶ Correctness: data race, starvation
- ▶ Example 2: two simultaneous money transfers
 - ▶ Correctness: deadlock
- ▶ Example 3: simple bank statistics
 - ▶ Efficiency: lack or dependency of work, overheads, ...

First example: simplified C code, not complete

Deposit/withdraw task

```
// code executed to process each transaction
// val contains the amount of money to deposit (positive) or to withdraw (negative)

#pragma omp task shared(acc) firstprivate(val)
if ((acc.balance + val) < 0)
    Error("Not enough money in account %d", acc.id);
else {
    acc.balance = acc.balance + val;
    Correct("New balance in account %d: %d\n", acc.id, acc.balance);
}
```

First example: two simultaneous withdraw operations

Figure 8: No problem if $id_1 \neq id_2$

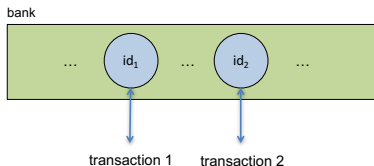
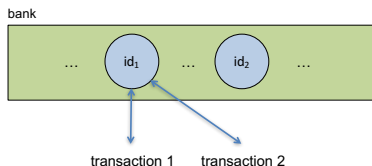


Figure 9: Concurrent execution of code on same account if $id_1 = id_2$:
data race



First example: data race – free money

Problem: **Data race** in the access to balance

Assume `acc.balance=105`, initially

Time	Transaction 1 (val1=-100)	Transaction 2 (val2=-10)
1	if ((acc.balance+val1)>0)	
2		if ((acc.balance+val2)>0)
	acc.balance=acc.balance+val1	acc.balance=acc.balance+val2
3	Step1: read acc.balance (105)	
4	Step2: sum = 105 + (-100)	
5		Step1: read acc.balance (105)
6	Step3: write acc.balance with 5	
7		Step2: sum = 105 + (-10)
8		Step3: write acc.balance with 95

Simplified C code, not complete

Using `omp_set_lock` and `omp_unset_lock` to protect the execution of account balance update

```
// code executed to process each transaction
#pragma omp task shared(acc) firstprivate(val)
if ((acc.balance + val) < 0)
    Error("Not enough money in account %d", acc.id);
else {
    omp_set_lock(&acc.lock);
    acc.balance = acc.balance + val;
    omp_unset_lock(&acc.lock);
    Correct("New balance in account %d: %d\n", acc.id, acc.balance);
}
```

First example: data race – money but negative balance

Problem: Still **data race** in the access to balance

Time	Transaction 1 (val1=-100)	Transaction 2 (val2=-10)
1	if ((acc.balance+val1)>0)	
2	Set lock	
	acc.balance=acc.balance+val1	
3	Step1: read acc.balance (105)	
4		if ((acc.balance+val2)>0)
5		Set lock failed
6	Step2: sum = 105 + (-100)	Set lock failed
7	Step3: write acc.balance with 5	Set lock failed
8	Unset lock	Set lock
		acc.balance=acc.balance+val2
9		Step1: read acc.balance (5)
10		Step2: sum = 5 + (-10)
11		Step3: write acc.balance with -5
12		Unset lock

Simplified C code, not complete

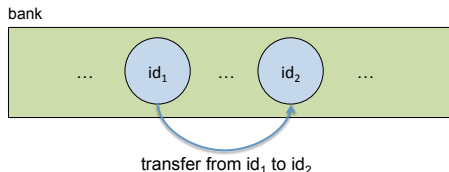
Using `omp_set_lock` and `omp_unset_lock` to protect the execution of account balance update

```
// code executed to process each transaction
#pragma omp task shared(acc) firstprivate(val)
{
    omp_set_lock(&acc.lock);
    if ((acc.balance + val) < 0)
        Error("Not enough money in account %d", acc.id);
    else {
        acc.balance = acc.balance + val;
        Correct("New balance in account %d: %d\n", acc.id, acc.balance);
    }
    omp_unset_lock(&acc.lock);
}
```


First example: correct execution

Time	Transaction 1 (val1=-100)	Transaction 2 (val2=-10)
1	Set lock	
2	if ((acc.balance+val1)>0)	
	acc.balance=acc.balance+val1	
3	Step1: read acc.balance (105)	
4	Step2: sum = 105 + (-100)	Set lock failed
5	Step3: write acc.balance with 5	Set lock failed
6	Unset lock	Set lock
7		if ((acc.balance+val2)>0)
8		Error: not enough money in account
9		Unset lock

Second example: transfer between two accounts

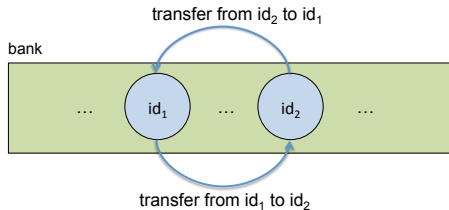


We need to protect the update of the two accounts

```
int transfer(account * from, account * to, int val) {  
    int status = 0;  
    omp_set_lock(&from->lock);  
    omp_set_lock(&to->lock)  
  
    if (from->balance > val) {  
        from->balance -= val;  
        to->balance += val;  
        status = 1;  
    }  
  
    omp_unset_lock(&to->lock);  
    omp_unset_lock(&from->lock);  
    return status;  
}
```

Second example: two simultaneous transfers, same account

But, what if "John wants to transfer \$10 to Peter's account" while
"Peter wants to also transfer \$20 to John's account"?



Both get blocked when invoking `omp_set_lock`

- Cycle in locking graph = **deadlock**

Second example: deadlock

Time	Transaction 1 (John to Peter)	Transaction 2 (Peter to John)
1	Set lock on John account	
2		Set lock on Peter account
3		Set lock on John account failed
4	Set lock on Peter account failed	
5	DEADLOCK	DEADLOCK

Second example: ordering lock acquisition

Standard solution: canonical order for locks (e.g. acquire in decreasing order)

```
int transfer(account * from, account * to, int val) {
    int status = 0;
    if (from->id > to->id) {
        omp_set_lock(&from->lock);
        omp_set_lock(&to->lock);
    } else {
        omp_set_lock(&to->lock);
        omp_set_lock(&from->lock);
    }

    if (from->balance > val) {
        from->balance -= val;
        to->balance += val;
        status = 1;
    }

    omp_unset_lock(&to->lock);
    omp_unset_lock(&from->lock);
    return status;
}
```

Other potential concurrency problems

Race Condition

- ▶ Multiple tasks read and write some data and the final result depends on the relative timing of their execution

Deadlock

- ▶ Two or more tasks are unable to proceed because each one is waiting for one of the others to do something

Starvation

- ▶ A task is unable to gain access to a shared resource and is unable to make progress

Livelock

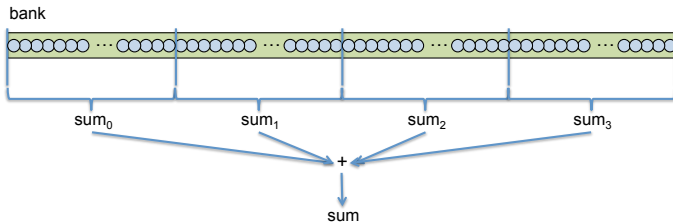
- ▶ Two or more tasks continuously change their state in response to changes in the other tasks without doing any useful work

Third example: bank statistics

- ▶ Imagine that every day the bank needs to compute the total interest (*sum*) that has to pay to all its customers (hundred thousands, or more!)
 - ▶ $sum = \sum_{i=1}^{number_clients} balance_i \times interest_i$
- ▶ For simplicity, if we assume that *balance* and *interest* are vectors with vector elements *i* associated to client *i*, then the computation of *sum* implies a **Dot Product** of two vectors:
 $sum = balance \times interest$

Third example: bank statistics

- ▶ The computation and data can be partitioned among multiple processors (P). each working with $1/P$ elements and accumulating the result in a "shared" variable (e.g. `sum`). This is shown in the following figure:



- ▶ Computation time approx. divided by P

Simplified C code, not complete

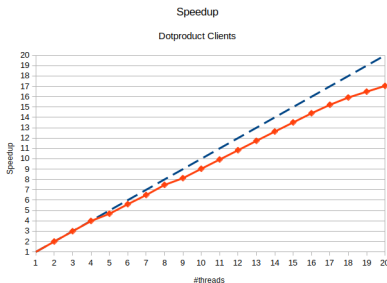
```
float balance[MAX_CLIENTS];
float interest[MAX_CLIENTS];

float DotProduct (float * balance, float * interest, long number_clients) {
    float sum = 0.0;
    // This distributes iterations among participating processors
    #pragma omp parallel for reduction(+: sum)
    for (int client = 0 ; client < number_clients; client++) {
        sum += balance[client] * interest[client];
    }
    return(sum);
}
```

Note: this is the so-called **work-sharing model** in OpenMP, only for loops; in this course we will be using the **tasking model** in OpenMP that applies to a wider set of code structures.

How much faster is the parallel version?

- ▶ **Parallel version** is almost P **times faster** than the **sequential version**



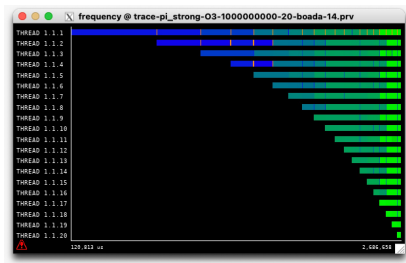
- ▶ Results are shown for
 - ▶ Boada (boada-12) machine using up to 20 cores of one node
 - ▶ A set of 100,000,000 clients
 - ▶ Dashed line shows ideal linear speedup

Potential parallelism problems

- ▶ Lack of work or work dependency
 - ▶ Coverage or extent of parallelism in algorithm
 - ▶ Dependencies (sequential is an extreme case)
 - ▶ Hard to equipartition the work
 - ▶ Load imbalance
 - ▶ Due to the parallelization strategy and parallel programming model
- ▶ Overheads of the parallelization
 - ▶ Granularity of partitioning among processors
 - ▶ Work generation and synchronization
 - ▶ Locality of computation and communication
- ▶ Frequency Scaling

Frequency Scaling

- High Frequency to Lower Frequency



- Analysis are shown for
 - Boada (boada-14) machine using up to 20 cores of one node
 - Computing pi

Outline

Motivation (video lesson 1)

Concurrency and parallelism

Examples and potential problems

Tasks, Threads, Processes and Processors

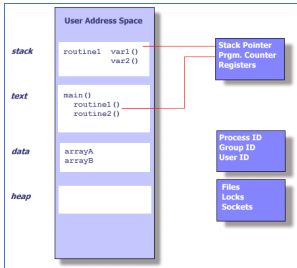
Tasks vs processes/threads vs Processors

- ▶ Tasks are created by the parallel runtime that supports the execution of a parallel language (e.g. `#pragma omp task`) and are not known by the OS (Operating System)
- ▶ Processes/threads are logical computing agents, offered by the OS, that execute tasks
- ▶ Processors² are the hardware units that physically execute those logical computing agents
- ▶ In most cases, there is a one-to-one correspondence between processes/threads and processors, but not necessarily (it is a OS decision)

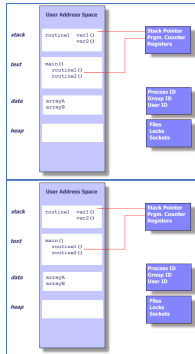
²CPU, processor and core refer to the same concept during this course and may be used interchangeably.

Processes vs. threads

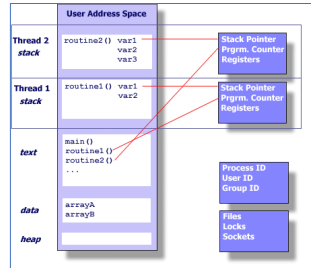
Sequential program (process)



Parallel program (processes)



Parallel program (threads)



**Processes do not share memory.
Threads do!**

Figure 10: Processes vs threads: User Address Space and other resources

Processes and threads

Processes and threads co-exist in the same parallel program:

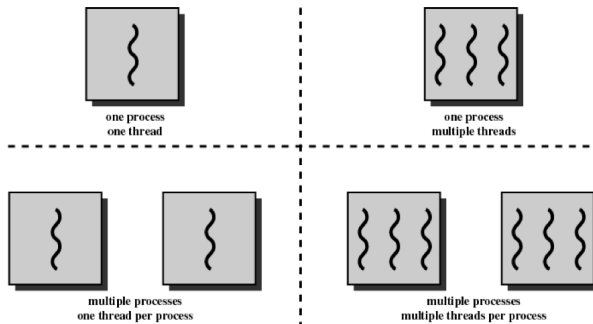


Figure 11: Processes and threads

Parallelism (PAR)

Unit 1: Why parallel computing?

Eduard Ayguadé,
Gladys Utrera (T20) and Daniel Jiménez (T10, T40)

Computer Architecture Department
Universitat Politècnica de Catalunya

Course 2025/26 (Spring semester)